

SERVICE ACTUEL · DATA-SERVE

Consultation, SQL, exposition et qualité des données

18 capacités analysées

services/data-serve · Kotlin 2.4.10, Spring Boot 4.1.1 et JVM ; PostgreSQL/JPA/Flyway ; Kafka/Protobuf ; Trino JDBC et JSqlParser ; MinIO/S3 pour les previews ; OAuth et client IAM.

Service de lecture pour l'interface, les clients OAuth et le MCP dataset. Il traduit les métadonnées métier en lectures Trino sur Iceberg, expose aperçus, lignes filtrées, SQL et téléchargements, et possède les définitions et calculs des règles qualité. Son nom historique apitoken ne signifie plus une gestion locale de clés API.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : Query, Catalog, Datasets, Audit, Egress, IAM, Flows, Applications, MCP, Maintenance, Migration.

Périmètre de lecture

- ServeController, ExpositionApiController et DataQualityController, leurs DTO et handlers domaine.
- QueryAnalyser, SQLQueryHandler, SqlFragmentGuard, TrinoJDBC, repositories de lecture et LosslessNdjson.
- Règles, historiques, comparateurs, templates, computes et événements de qualité.
- Authentification, résolution MCP, client IAM, projections Kafka, migration de schéma, outbox/dead letters, configuration Trino et charts.

Objets et cycle de vie

- Dataset et DatasetBranch sont des projections de data-model, avec types de colonnes, labels, état, sensibilité, qualité et owner. Elles ne constituent pas actuellement un registre d'éditions Iceberg.
- Query, Statement et ExecutionContext portent le SQL et les datasets détectés. DataFrame, Table, TabularSchema et Pagination décrivent la réponse ; aucune session durable de pagination n'est présente dans les contrats lus.
- DatasetContent possède un Stream de lignes et le nom/MIME de fichier. Les frames NDJSON conservent schéma, valeurs positionnelles, compteur et digest, sans constituer une publication de dataset.
- Rule est l'identité d'une règle avec suppression logique ; RuleHistory versionne le script et les seuils. Compute est lié à une exécution de dataset et ComputedRule porte une valeur typée et son résultat.
- PreviewScope porte le projet et les datasets/branches d'entrée d'une version. Le fichier preview reste dans le stockage objet et reçoit les droits de ce scope.
- Les tables utilisateur, labels et liens de projets sont des miroirs. Outbox et dead letters gardent les événements et leur état de traitement.

Dépendances du service

- data-model alimente datasets, branches, schémas, labels et liens sur Kafka/Protobuf. data-serve publie règles et computes en retour.
- process-data publie les événements de fin d'exécution qui déclenchent les contrôles qualité sur main.

- Trino JDBC lit les lignes et schémas Iceberg, résout les refs, exécute les SQL utilisateurs et règles, et fournit les types des exports. Retirer seulement Spring ne retire donc pas la JVM du parcours.
- MinIO/S3 fournit les fichiers preview. data-dev expose leur scope de projet et d'entrées via HTTP interne.
- IAM HTTP arbitre CONSULT/DOWNLOAD/PROJECT et valide l'acteur MCP ; Keycloak fournit actuellement les JWT et tokens techniques. La bibliothèque partagée conserve des résolutions d'identité en cache.
- PostgreSQL conserve projections, règles, histories, computes, outbox, dead letters et état de migration. Notification-center reçoit les abonnements qualité.

Fonctionnalités et compatibilité à préserver

DS-01 Cible non qualifiée

Construire la projection de serving depuis les faits du catalogue

COMPORTEMENT OBSERVÉ

Le consommateur data-model crée ou remplace la projection locale du dataset et de ses branches. UPDATED est un upsert utilisable même sans CREATE antérieur ; il exige un snapshot after dont l'ID correspond à l'agrégat. DELETE supprime une projection existante et tolère son absence. Les liens de projet ne sont conservés que pour les états ACCEPTED ou OWNED ; demandes et refus suppriment un ancien lien pendant le jeu. Les labels sont répliqués séparément. Les utilisateurs ont un miroir local alimenté par un parcours IAM et un listener historique encore présent sans condition sur LegacyUserCreatedConsumer.

UTILISATEURS ET RÔLES

Consommateurs Kafka et toutes les lectures de data-serve.

INTERFACES DANS LE CODE

DataModelEventKafkaConsumer.consumeCreateDataset ; consumeUserCreated pour les liens ; DatasetLabelEventKafkaConsumer ; LegacyUserCreatedConsumer.

DONNÉES ET ÉTATS

Dataset, DatasetBranch, schémas, liens projet, labels et user_table_serve PostgreSQL ; aucune EditionRef dans l'agrégat actuel.

DÉPENDANCES ET ÉCHANGES

data-model-dataset, data-model-project-dataset-link, data-model-label et événements d'identité ; Kafka au moins une fois.

COMPORTEMENT CIBLE PROPOSÉ

Catalog Rust publie une projection versionnée et Query conserve seulement les index nécessaires à la résolution. Les lignes restent lues depuis des éditions approuvées. Chaque fait comporte ID, agrégat et révision ; un événement ancien ne peut ressusciter une projection supprimée.

MIGRATION DE CETTE CAPACITÉ

Rejouer le catalogue dans une projection parallèle puis comparer les UUID, branches, labels et liens accordés. Conserver la compatibilité Protobuf pendant le basculement. Vérifier la coexistence des deux sources utilisateur au lieu de supposer que le drapeau IAM arrête le listener historique.

CRITÈRE D'ACCEPTATION À EXÉCUTER

UPDATED reçu avant CREATE établit la projection. Un refus rejoué retire un ancien lien autorisé. Une suppression puis un événement ancien ne rend pas le dataset visible. La perte de la projection se répare sans nouvelle publication de données.

Références de l'analyse

[monorepo · services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/event/DataModelEventKafkaConsumer.kt:32](#) Upsert et contrôles du snapshot reçu.

[monorepo · services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/event/DataModelEventKafkaConsumer.kt:85](#) Seuls les liens accordés restent dans la projection.

[monorepo · services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/user/event/LegacyUserCreatedConsumer.kt:18](#) Listener historique non conditionné dans ce fichier.

[monorepo · services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/aggregate/Dataset.kt:128](#) Agrégat de serving sans référence d'édition.

DS-02 **Cible non qualifiée**

Authentifier les lecteurs OAuth et préserver l'acteur des appels MCP

COMPORTEMENT OBSERVÉ

Les routes natives `/api/v1` et `/internal/mcp` utilisent une chaîne Bearer obligatoire. Le reste peut accepter X-Userinfo uniquement sous configuration gateway, tout en conservant le resource server Bearer. L'interceptor résout l'utilisateur, exige qu'il soit actif et ignore les anciens en-têtes de clés API. Le MCP exige le client `cleyrop-mcp-dataset` et demande à IAM de résoudre acteur et contexte à partir du Bearer de transport ; l'acteur retourné doit correspondre à celui demandé. Le compte de transport n'accorde pas lui-même la lecture des données.

UTILISATEURS ET RÔLES

Utilisateur web, client OAuth, outil MCP opérant pour un utilisateur.

INTERFACES DANS LE CODE

ExpositionApiController ; TokenHandlerInterceptor ; McpReadIdentityResolver ; POST IAM `/api/v1/internal/mcp/read-identity`.

DONNÉES ET ÉTATS

Identité de requête et miroir utilisateur ; cache de résolution IAM et cache du token de service, pas clé API locale.

DÉPENDANCES ET ÉCHANGES

Fournisseur OIDC actuellement Keycloak, gateway APISIX, IAM et `cleyrop-spring-auth`.

COMPORTEMENT CIBLE PROPOSÉ

IAM/Query Rust valident le jeton du transport et le sujet/acteur/context distincts. Ils appliquent les audiences et destinations prévues sans dépendre de Keycloak. MCP reçoit une capacité de lecture limitée aux éditions et opérations autorisées, jamais une clé globale.

MIGRATION DE CETTE CAPACITÉ

Conserver les familles de routes et les enveloppes d'erreur. Porter les mappings subject vers IAM ID, puis supprimer le couplage JVM et Keycloak. Tester les chemins directs MCP et gateway séparément, y compris les protections réseau.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un en-tête historique `token/apikey` n'authentifie pas un appel. Le bon compte MCP avec le mauvais acteur ou contexte est refusé. Une identité inactive ne peut ni lister ni lire, même si une résolution précédente est en cache.

Références de l'analyse

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/configuration/WebSecurityConfiguration.kt:43](#) Chaîne Bearer native et chaîne gateway distinctes.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/configuration/interceptor/TokenHandlerInterceptor.kt:26](#) Résolution et activité utilisateur.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/authz/McpReadIdentityResolver.kt:28](#) Validation du transport et de l'acteur auprès d'IAM.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/configuration/CurrentUserResolver.kt:25](#) Cache de résolution d'identité.

DS-03 **Cible non qualifiée****Déléguer consultation et téléchargement à IAM****COMPORTEMENT OBSERVÉ**

Le service construit des faits sur dataset, projet propriétaire, projets liés, diffusion et sensibilité de la branche, puis interroge le broker IAM. Les opérations CONSULT et DOWNLOAD sont distinctes, ainsi que l'accès PROJECT. Les liens sont découpés par groupes de douze et les demandes par cent contrôles ; les réponses d'un dataset sont combinées seulement après obtention complète. Une erreur, réponse vide, booléen mal typé ou vecteur incomplet produit une indisponibilité, sans politique de secours locale. Une erreur 401 du broker renouvelle une fois le token de service. /permissions ne retourne actuellement que canDownload.

UTILISATEURS ET RÔLES

Tout lecteur de lignes ou de fichiers et interface qui prépare le téléchargement.

INTERFACES DANS LE CODE

GET /datasets/{datasetId}/permissions ; DatasetAuthorizationService ; POST IAM /api/v1/internal/data-serve/authorize.

DONNÉES ET ÉTATS

Projection de faits du dataset et liens accordés ; décisions par appel ; cache du token technique.

DÉPENDANCES ET ÉCHANGES

IAM/OpenFGA, miroir user pour retrouver le subject, gateway pour le contrôle de route.

COMPORTEMENT CIBLE PROPOSÉ

Query Rust applique une autorisation centralisée à l'édition, au schéma et à l'opération, avec policy revision et identité complètes. Les pages et les effets d'export réévaluent la révocation selon le contrat ; aucun cache de résultat ne contourne IAM.

MIGRATION DE CETTE CAPACITÉ

Porter les limites de lots et l'échec fermé. Stabiliser les identités sans dépendre d'une projection utilisateur périmée. Étendre le contrat à tenant, édition et droits de colonnes/lignes seulement avec une politique spécifiée et qualifiée.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un dataset avec plus de douze projets liés reste lisible si un groupe l'autorise et tous les contrôles répondent. Si un autre groupe échoue, aucun octet ne sort. Le bouton téléchargement et le download réel utilisent exactement la même décision.

Références de l'analyse

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/authz/DatasetAuthorizationService.kt:45](#) Lots, combinaison des droits et échec fermé.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/authz/IamDatasetAuthorizationClient.kt:32](#) Réponse complète, retry 401 et décodage strict.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/query/DatasetPermissionsQueryHandler.kt:31](#) canDownload vient du même arbitre que le téléchargement.

DS-04 **Cible non qualifiée**

Lister les datasets réellement consultables

COMPORTEMENT OBSERVÉ

L'API d'exposition liste les datasets accessibles, avec filtres qualité, sensibilité, état, propriétaires, description, nom affiché et nom du propriétaire ; ownerId prime sur ownerDisplayName. includeColumns ajoute les colonnes et descriptions. Le handler parcourt les candidats par lots de cent, interroge IAM sur chaque branche main puis conserve tous les autorisés en mémoire avant la pagination demandée. Les totaux décrivent donc les datasets consultables. Le coût croît avec l'ensemble des candidats et des décisions IAM, même pour une petite première page.

UTILISATEURS ET RÔLES

Client d'intégration, MCP dataset et applications de lecture.

INTERFACES DANS LE CODE

GET /api/v1/datasets ; GET /internal/mcp/v1/datasets.

DONNÉES ET ÉTATS

Projection locale des datasets et utilisateurs ; collection de candidats puis liste autorisée.

DÉPENDANCES ET ÉCHANGES

IAM batch ; PostgreSQL de serving ; labels/schémas de data-model.

COMPORTEMENT CIBLE PROPOSÉ

Catalog/Query Rust conserve une liste filtrée par droits et une pagination bornée. Le plan de recherche doit éviter de charger tous les objets pour chaque page, sans produire de totaux qui révèlent des objets interdits. Les droits sont revérifiés à l'ouverture de lecture.

MIGRATION DE CETTE CAPACITÉ

Comparer champs, ordre, includeColumns, pagination externe à base un et types d'erreur. Mesurer le coût sur le volume réel du catalogue avant de choisir index d'autorisation, curseur ou total approximatif explicitement annoncé.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une liste de cent candidats dont trente sont autorisés annonce trente, sans IDs des autres. La priorité ownerId est conservée. Une révocation entre liste et lecture bloque la lecture et la charge mémoire reste bornée sur un gros catalogue.

Références de l'analyse

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/apitoken/controller/ExpositionApiController.kt:182](#) Paramètres et alias MCP de la liste.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/apitoken/query/ListDatasetApiTokenQueryHandler.kt:43](#) Résolution des propriétaires puis filtrage de tous les candidats.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/apitoken/query/ListDatasetApiTokenQueryHandler.kt:65](#) Coût de parcours complet avant pagination.

DS-05 Cible non qualifiée

Lire une table par UUID ou nom avec projection et filtres bornés

COMPORTEMENT OBSERVÉ

Les lectures simples et structurées résolvent un UUID ou un techName, prennent main, vérifient CONSULT, refusent CREATED/FAILURE et valident `page>=1` et `size` dans la borne configurée. `select` est une liste de colonnes de ce dataset ; `where` est reconstruit depuis une grammaire admise de comparaisons, AND/OR/NOT, NULL, BETWEEN, IN et LIKE/ILIKE. Fonctions, casts, sous-requêtes, commentaires et séparateurs de statements sont refusés. Les fragments ont une limite de longueur, profondeur et nombre de valeurs. `orderBy` accepte colonnes retournées ou positions, direction et NULLS FIRST/LAST ; un départage par positions complète le tri.

UTILISATEURS ET RÔLES

Applications, intégrations et lecteurs MCP sans éditeur SQL complet.

INTERFACES DANS LE CODE

GET `/api/v1/datasets/{dataset}` ; GET `/api/v1/datasets/structured/{dataset}` ; mêmes chemins sous `/internal/mcp/v1`.

DONNÉES ET ÉTATS

Réponse JSON brute ou structurée avec schéma et pagination ; lecture de snapshot Trino résolu pour main.

DÉPENDANCES ET ÉCHANGES

IAM CONSULT, schéma répliqué, SqlFragmentGuard et Trino.

COMPORTEMENT CIBLE PROPOSÉ

Query Rust expose le même sous-ensemble via un AST typé sans concaténation de fragments clients. Il lie noms et field IDs à une édition autorisée et versionne la sérialisation des nombres, dates, binaires et valeurs complexes. Une lecture sans schéma fiable doit obtenir le schéma autoritaire de l'édition.

MIGRATION DE CETTE CAPACITÉ

Réutiliser un corpus de sélections et filtres valides/invalides, y compris colonnes accentuées et noms réservés. Préserver les formats de réponse historiques dans la façade. Remplacer le fallback actuel d'identifiants simples sur schéma vide par une résolution autoritaire.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une injection dans `select/where/orderBy` n'atteint pas le moteur. Une projection de deux colonnes ne permet pas de trier par une troisième cachée. Une page avec valeurs nulles, binaire, date et struct garde les types prévus par le contrat.

Références de l'analyse

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/apitoken/query/ReadDatasetApiTokenQueryHandler.kt:99](#) Résolution, droits, états et pagination.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/common/SqlFragmentGuard.kt:41](#) Grammaire admise et reconstruction.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/common/SqlFragmentGuard.kt:124](#) Bornes des fragments.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/repository/TableRepositoryTrinoImpl.kt:138](#) Lecture de page et tri stable.

DS-06 Cible non qualifiée

Retourner les métadonnées métier du dataset exposé

COMPORTEMENT OBSERVÉ

L'endpoint de métadonnées résout UUID ou nom technique, exige inProduction et une branche main, puis CONSULT. Il retourne type d'origine, date de création, description, qualité, sensibilité, dernier refresh, nom du propriétaire, état, fréquence, append/override, mode de schéma, diffusion et noms des labels. Les métadonnées sont issues de la projection PostgreSQL et ne portent ni snapshot ID ni version de contrat ni preuve d'approbation.

UTILISATEURS ET RÔLES

Intégrateur, application et MCP souhaitant décrire les données avant lecture.

INTERFACES DANS LE CODE

GET /api/v1/datasets/{dataset}/metadata ; GET /internal/mcp/v1/datasets/{dataset}/metadata.

DONNÉES ET ÉTATS

DatasetMetadataView construit depuis dataset/main, utilisateur et labels.

DÉPENDANCES ET ÉCHANGES

Projections data-model/utilisateurs ; IAM CONSULT.

COMPORTEMENT CIBLE PROPOSÉ

Catalog Rust renvoie les métadonnées métier avec une EditionRef et la révision de politique. Le propriétaire et les tags actuels sont distingués des métadonnées de l'édition historique. Le kind est explicite et ne reprend pas le type de connecteur.

MIGRATION DE CETTE CAPACITÉ

Maintenir les noms de champs et valeurs enum dans la façade. Ajouter des champs d'édition optionnels puis une version d'API qui les impose. Mesurer et rendre visible la fraîcheur de la projection.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un dataset hors production reste introuvable via ce parcours. Une EditionRef obtenue avec les métadonnées permet une lecture répétable ; une nouvelle collecte ne change pas les données de cette référence. Les droits sont vérifiés même si les métadonnées sont en cache.

Références de l'analyse

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/query/GetDatasetMetadataQueryHandler.kt:28](#) Champs réellement exposés.

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/query/GetDatasetMetadataQueryHandler.kt:61](#) Production, branche, droits, propriétaire et labels.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/apitoken/controller/ExpositionApiController.kt:398](#) Route et alias MCP.

DS-07 **Cible non qualifiée**

Afficher un aperçu de dataset

COMPORTEMENT OBSERVÉ

Le preview d'un dataset résout la branche demandée ou main. Une branche CREATED ou FAILURE retourne aucun aperçu. Après autorisation CONSULT, le repository lit par défaut cinquante lignes via Trino et assemble les valeurs avec le schéma répliqué et la date de refresh. WARNING reste lisible. Les erreurs de branche manquante et dataset manquant sont distinctes dans le domaine. La route rend 404 lorsqu'aucun DataFrame n'est produit.

UTILISATEURS ET RÔLES

Utilisateur du catalogue et de l'éditeur qui inspecte un échantillon.

INTERFACES DANS LE CODE

GET /datasets/{datasetId}/preview?branchName=... ; PreviewDatasetQueryDataset.

DONNÉES ET ÉTATS

DataFrame contenant Table, TabularSchema et refreshedDate ; projection de branche.

DÉPENDANCES ET ÉCHANGES

IAM CONSULT ; Trino/Iceberg ; data-model pour schéma et état.

COMPORTEMENT CIBLE PROPOSÉ

Query Rust fournit un échantillon borné d'une édition explicitement autorisée, avec schema ID, type précis et budget d'octets. Le résultat cite l'édition plutôt que la seule date d'exécution. Les erreurs d'une tentative récente ne doivent pas masquer une édition approuvée antérieure sans règle métier explicite.

MIGRATION DE CETTE CAPACITÉ

Comparer valeurs et typage sur les cinquante premières lignes des corpus de référence. Décider et versionner l'ordre de l'aperçu, qui n'est pas un ordre métier garanti actuellement. Revoir l'effet de FAILURE lorsque des données approuvées existent déjà.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un aperçu d'une édition répétée reste cohérent après refresh. L'utilisateur sans CONSULT ne reçoit ni lignes ni types protégés. Une valeur très volumineuse ne dépasse pas le budget du preview.

Références de l'analyse

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/controller/ServeController.kt:122](#) Contrat HTTP et branche optionnelle.

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/query/PreviewDatasetQueryHandler.kt:109](#) État, droits et assemblage.

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/repository/TableRepository.kt:21](#) Limite par défaut de cinquante lignes.

DS-08 **Cible non qualifiée**

Consulter les résultats intermédiaires des transformations low-code

COMPORTEMENT OBSERVÉ

POST /version/preview prend version et transformation. data-serve demande à data-dev le projet et toutes les entrées de la version, vérifie l'identité de la version retournée et la forme des branches, puis exige accès au projet et CONSULT sur toutes les entrées. Une version sans entrée reste régie par le projet. Il lit ensuite un fichier preview.json dans le stockage MinIO/S3, indexé par version et transformation. Un scope absent ou une entrée introuvable produit aucun preview ; une autorité scope indisponible produit une indisponibilité explicite.

UTILISATEURS ET RÔLES

Auteur ou lecteur de flow et interface low-code.

INTERFACES DANS LE CODE

POST /version/preview ; GET interne /api/v1/internal/dataflow-versions/{id}/preview-scope appelé vers data-dev.

DONNÉES ET ÉTATS

PreviewScope projet/inputs et objet S3 intermédiaire ; pas d'édition publiée de dataset.

DÉPENDANCES ET ÉCHANGES

data-dev HTTP avec identité de service ; IAM ; MinIO/S3 ; artefacts process-data.

COMPORTEMENT CIBLE PROPOSÉ

Flows et Query Rust lient chaque preview à run, transformation, révision de plan et éditions d'entrée. Les artefacts sont privés, immuables, bornés et munis d'une rétention. Leur contrôle de lecture utilise sujet, acteur, projet et toutes les entrées, même après retrait d'un partage.

MIGRATION DE CETTE CAPACITÉ

Importer le mapping des versions vivantes et conserver la lecture des anciens artefacts jusqu'à leur expiration. Remplacer les chemins conventionnels par un manifeste de preview dont le producteur est authentifié. Éviter qu'un simple UUID d'objet fournisse le droit de lecture.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un utilisateur autorisé sur le projet mais interdit sur une entrée ne lit pas le preview. Une version inconnue et une entrée supprimée n'exposent pas d'informations supplémentaires. Un preview généré sans entrée reste limité aux droits du projet.

Références de l'analyse

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/query/PreviewDatasetQueryHandler.kt:66](#) Vérification du scope et des entrées avant lecture.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/repository/DataDevPreviewScopeClient.kt:41](#) Autorité distante, validation stricte et erreurs.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/repository/TableRepositoryTrinoImpl.kt:241](#) Lecture du preview stocké en objet.

Exécuter du SQL de lecture avec analyse des relations

COMPORTEMENT OBSERVÉ

L'éditeur SQL accepte un projet et une branche facultative ; l'exposition propose SQL global ou par projet, toujours sur main. Query découpe les statements sur les points-virgules et n'en accepte qu'un. JSqlParser extrait opérations et relations ; seules SELECT et EXPLAIN passent, les syntaxes non parsables échouent. Les fonctions de table sont refusées sauf UNNEST. Le service vérifie l'accès au projet puis à chaque dataset et, en contexte projet, son appartenance ou lien. L'éditeur permet une requête sans dataset après contrôle du projet ; exposition/MCP la refusent. Il n'y a pas de CRUD SQL d'écriture dans ces contrôleurs.

UTILISATEURS ET RÔLES

Analyste SQL, application OAuth et MCP dataset.

INTERFACES DANS LE CODE

POST /projects/{projectID}/datasets/sql/query ; POST /api/v1/projects/{projectID}/datasets/sql/query ; POST /api/v1/datasets/sql/query ; POST /internal/mcp/v1/datasets/sql/query.

DONNÉES ET ÉTATS

ExecutionContext transitoire, statement, opérations, noms de datasets et inputIds dans la réponse.

DÉPENDANCES ET ÉCHANGES

IAM CONSULT/PROJECT, liens de projet, JSqlParser et Trino JDBC.

COMPORTEMENT CIBLE PROPOSÉ

Query Rust analyse, lie puis exécute un AST sur des EditionRef approuvées avec Icegres ou un moteur sans JVM qualifié. Les capacités dialectales sont explicites. L'analyse de sécurité doit porter sur tout le plan, sous-requêtes, fonctions et EXPLAIN compris, avec aucune résolution physique hors liste autorisée.

MIGRATION DE CETTE CAPACITÉ

Créer une matrice de compatibilité SELECT, CTE, joins, agrégations, types, fonctions, EXPLAIN et erreurs. Rejouer un corpus expurgé et comparer résultats. Remplacer le split textuel et les réécritures regex par un parseur et binder communs.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une jointure exige le droit sur chaque entrée et un contexte projet cohérent. TABLE d'accès indirect est refusé avant moteur. Un point-virgule dans un littéral suit le dialecte documenté. Une requête sans dataset conserve la différence éditeur/exposition tant que la façade la promet.

Références de l'analyse

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/aggregate/Query.kt:6](#) Découpage actuel des statements.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/service/QueryAnalyserImplJSQL.kt:20](#) Extraction des relations et refus des fonctions de table.

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/service/QueryAnalyser.kt:113](#) Droits sur projet et chaque dataset.

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/query/SQLQueryHandler.kt:148](#) Distinction éditeur et exposition.

Épingler les snapshots et paginer les résultats SQL

COMPORTEMENT OBSERVÉ

TrinoJDBC lit `snapshot_id` depuis la table de refs pour la branche, puis utilise `FOR VERSION AS OF`. La lecture de dataset résout le snapshot pour chaque appel. Le SQL arbitraire résout chaque entrée séparément et réécrit le statement par regex ; un snapshot manquant laisse actuellement le nom sans clause. Le SQL paginé interroge d'abord le schéma, enveloppe la requête avec `COUNT OVER`, tri et offset/limit, puis conserve total et `hasMore`. Si une page dataset est vide au-delà du début, le comptage de secours résout de nouveau la ref. Aucune session de pagination exposée ne fixe la même édition sur plusieurs pages ou plusieurs tables.

UTILISATEURS ET RÔLES

Clients paginés, analystes et équipes qui exigent une lecture cohérente.

INTERFACES DANS LE CODE

`SQLQueryPaged` ; `TrinoJDBC.queryForListWithTotal/queryForRowSetPaged` ; réponses Pagination.

DONNÉES ET ÉTATS

Snapshot IDs transitoires ; tri dérivé des colonnes sélectionnées ; aucun cursor d'édition persistant dans le contrat actuel.

DÉPENDANCES ET ÉCHANGES

Trino et refs Iceberg ; schéma de résultat JDBC ; politique de pages du service.

COMPORTEMENT CIBLE PROPOSÉ

Query Rust résout un `ReadSet` avant exécution. Ce `ReadSet` contient table UUID, édition/snapshot, schema/spec et policy revision pour toutes les entrées. Les pages reprennent un curseur signé lié à ce `ReadSet` et au sujet ; une release multi-sorties fournit un ensemble atomique de participants. Une ref absente ne retombe jamais sur l'état courant.

MIGRATION DE CETTE CAPACITÉ

Conserver la pagination numérique dans la façade mais lui associer un handle stable. Réécrire le plan au niveau AST/scan, sans regex. Ajouter des scénarios de branches absentes, noms quotés, ref déplacée, lignes identiques et publication concurrente.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Publier entre page 1 et page 2 ne produit ni saut ni mélange d'éditions. Une requête multi-sorties lit tous les membres de la même release. Une ref inconnue échoue avant scan. Le total d'une page vide est calculé sur le même snapshot.

Références de l'analyse

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/repository/jpa/TrinoJDBC.kt:69](#) Résolution de ref par branche.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/repository/jpa/TrinoJDBC.kt:122](#) Page avec total et recomptage.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/repository/jpa/TrinoJDBC.kt:254](#) Plan de pagination SQL.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/repository/jpa/TrinoJDBC.kt:473](#) Réécriture regex et absence de clause si snapshot introuvable.

Télécharger un dataset ou une requête en CSV

COMPORTEMENT OBSERVÉ

Le download dataset prend branche, quoted, separator et format. Pour CSV, il exige DOWNLOAD, refuse CREATED/FAILURE, écrit une ligne d'en-tête puis les lignes en streaming. Le download SQL réutilise l'analyse de requête avec l'opération DOWNLOAD pour toutes les relations et refuse une requête sans dataset. Le résultat porte un nom de fichier et Content-Disposition. Le corps ferme le Stream après transfert. Le pool Trino de download est séparé du pool de lecture pour qu'un client lent n'accapare pas les connexions interactives ; sa saturation aboutit à un timeout borné et une erreur 503.

UTILISATEURS ET RÔLES

Utilisateur qui extrait des données et intégrateur consommant un dump.

INTERFACES DANS LE CODE

GET /datasets/{datasetId}/download ; POST /datasets/download ; POST /api/v1/datasets/download.

DONNÉES ET ÉTATS

Stream HTTP/JDBC, filename et paramètres CSV ; aucun reçu d'export durable ni destination externe persistée.

DÉPENDANCES ET ÉCHANGES

IAM DOWNLOAD ; Trino download pool ; MVC StreamingResponseBody.

COMPORTEMENT CIBLE PROPOSÉ

Query/Egress Rust produit un flux borné à partir d'une EditionRef approuvée, avec limites de durée, volume, concurrence et annulation. Un EXPORT vers un tiers est un effet distinct avec destination déclarée, clé d'idempotence et reçu. Le résultat peut être préparé en objet privé puis téléchargé sous droit réévalué.

MIGRATION DE CETTE CAPACITÉ

Conserver séparateur, quoting, en-têtes, charset, nom de fichier et comportements null/types dans la façade. Qualifier l'échappement CSV sur textes contenant séparateur et retour ligne. Décider le comportement d'une révocation pendant un transfert long.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un utilisateur qui peut consulter mais ne peut télécharger ne reçoit aucun octet. Une jointure nécessite DOWNLOAD sur chaque source. Annuler un transfert libère le scan et les ressources. Plusieurs clients lents n'empêchent pas les previews de répondre.

Références de l'analyse

[monorepo · services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/query/DownloadDatasetQueryHandler.kt:62](#) Droit de téléchargement, états et formats.

[monorepo · services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/query/DownloadDatasetQueryHandler.kt:89](#) SQL export avec droit DOWNLOAD.

[monorepo · services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/repository/TableRepositoryTrinoImpl.kt:195](#) Production CSV en flux.

[monorepo · services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/controller/CsvStreamingResponse.kt:6](#) Fermeture du flux et écriture HTTP.

[monorepo · services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/configuration/TrinoConfiguration.kt:24](#) Isolation des connexions de téléchargement.

DS-12 **Cible non qualifiée**

Exporter les types sans perte via le protocole NDJSON

COMPORTEMENT OBSERVÉ

Le format NDJSON du download dataset n'est pas une simple liste d'objets métier. Il commence par un frame schema portant noms et signatures de types, continue avec des frames row positionnels et termine par rowCount et SHA-256. Les décimaux et flottants sont encodés en texte, le binaire en base64, les dates/temps en forme textuelle. Les ROW restent positionnels pour ne pas perdre les noms de champs dupliqués ; les maps deviennent des paires key/value. L'imbrication est bornée. Le digest couvre les octets des frames row encodés, sans les retours ligne ajoutés par le contrôleur, ni les frames schema/end.

UTILISATEURS ET RÔLES

Client de copie, migration de dataset et intégration qui doit préserver les types.

INTERFACES DANS LE CODE

GET /datasets/{datasetId}/download?format=NDJSON ; DetailedRowStream.toLosslessNdjson.

DONNÉES ET ÉTATS

Frames schema/row/end ; compteur et digest calculés pendant le flux ; schema et lignes épinglés au même snapshot dans ce parcours.

DÉPENDANCES ET ÉCHANGES

Trino JDBC pour signatures et valeurs typées ; IAM DOWNLOAD ; lecteur de frames compatible.

COMPORTEMENT CIBLE PROPOSÉ

Query/Egress Rust conserve ce protocole versionné ou fournit Arrow IPC avec un adaptateur compatible. Le digest, les bornes et la représentation des types font partie du contrat. Un consommateur rejette un flux tronqué sans end et ne publie pas une copie partielle.

MIGRATION DE CETTE CAPACITÉ

Documenter le checksum exact avant portage. Comparer valeurs complexes, NaN/infinis, grands entiers/décimaux, timezone, maps à clés non textuelles et ROW avec noms dupliqués. Prévoir une négociation de version de format.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un aller-retour préserve null, précision décimale, timezone, binaire, ordre des champs et clés de map. Couper le transfert avant end empêche le commit du destinataire. Le digest recalculé selon la définition correspond exactement au frame final.

Références de l'analyse

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/repository/LosslessNdjson.kt:23](#) Frames, compteur et digest.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/repository/LosslessNdjson.kt:57](#) Encodage des types et limite de profondeur.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/repository/jpa/TrinoJDBC.kt:190](#) Schéma détaillé et lignes sur le même snapshot.

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/query/DownloadDatasetQueryHandler.kt:81](#) Choix NDJSON dans le contrat.

DS-13 **Cible non qualifiée**

Créer, versionner et supprimer les règles qualité

COMPORTEMENT OBSERVÉ

Une Rule identifie le dataset, l'auteur et sa suppression logique ; chaque création ou modification écrit une RuleHistory avec nom, description, script et seuils. Le nom actif est unique sans distinction de casse. Le service vérifie comparateurs, champs non blancs, existence du dataset, syntaxe SQL et exécution de la requête avant sauvegarde. Les comparateurs varient par type boolean, numeric, instant ou string et sont ordonnés ; la première condition satisfaite détermine le niveau d'alerte. Les modifications et lectures portant ruleId passent aussi datasetId afin qu'une règle d'un autre dataset soit introuvable. isNotify crée une souscription d'échec qualité.

UTILISATEURS ET RÔLES

Éditeur de règles et responsable qualité.

INTERFACES DANS LE CODE

POST /datasets/{datasetId}/rules ; PUT/DELETE /datasets/{datasetId}/rules/{ruleId} ; GET règles, règle, check du nom et history.

DONNÉES ET ÉTATS

Rule, RuleHistory, suppression logique, auteur, script et niveaux CRITICAL/MAJOR/MINOR.

DÉPENDANCES ET ÉCHANGES

Gateway IAM can_edit_rules/can_view_rules ; Trino pour validation ; outbox vers data-model ; notification-center.

COMPORTEMENT CIBLE PROPOSÉ

Audit Rust conserve une définition versionnée de règle et un contrat de résultat. Les seuils numériques utilisent un type exact adapté, sans conversion systématique vers Double. La règle est compilée et autorisée sur toutes ses entrées, puis exécutée sur un candidat ou une édition précise.

MIGRATION DE CETTE CAPACITÉ

Importer toutes les versions et tombstones. Garder l'historique des valeurs et comparateurs, y compris les limitations de conversion du type instant et numeric. Distinguer validation syntaxique, test utilisateur et audit bloquant de publication.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Modifier une règle crée une version nouvelle sans altérer les preuves d'une édition ancienne. Un ruleId d'un autre dataset est introuvable sur toutes les routes. Un seuil numérique supérieur à 2^{53} garde sa valeur exacte. Deux créations concurrentes du même nom ne réussissent pas ensemble.

Références de l'analyse

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataquality/command/SaveRuleCommandHandler.kt:86](#) Validations, unicité et exécution avant sauvegarde.

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataquality/command/SaveRuleCommandHandler.kt:172](#) Versions créées/modifiées et notifications.

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataquality/aggregate/RuleType.kt:59](#) Ordre et arité des comparateurs.

[monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataquality/aggregate/RuleType.kt:189](#) Conversion numérique actuelle en Double.

monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataquality/controller/DataQualityController.kt:64 Binding ruleId/datasetId.

DS-14 Cible non qualifiée

Fournir les modèles et l'essai interactif des règles qualité

COMPOTEMENT OBSERVÉ

Quinze templates couvrent volumes, duplicats de lignes, unicité de colonnes, combinaisons distinctes, distribution, intervalles, email, appartenance à une liste, minimum, maximum, médiane, moyenne et non-null. Ils retournent du SQL avec le nom technique ou un placeholder. Le check exécute le script sur main et exige une seule colonne, une seule ligne et une valeur non nulle d'un type admis. Le contrôle de statement exige SELECT et la présence du dataset ciblé, mais ne vérifie pas que l'ensemble des relations se réduit à ce dataset et ne demande pas CONSULT sur d'autres relations. Certains templates ne correspondent pas à leur libellé, par exemple maximum inférieur utilise $\geq$ et médiane égale utilise $\geq$.

UTILISATEURS ET RÔLES

Utilisateur qui prépare une règle avant enregistrement.

INTERFACES DANS LE CODE

GET /datasets/{datasetId}/rules/templates/{templateType} ; POST /datasets/{datasetId}/rules/check.

DONNÉES ET ÉTATS

Résultat transitoire type/valeur ou erreur métier ; aucun compute persistant pour ce check.

DÉPENDANCES ET ÉCHANGES

JSqlParser, schéma/techName local et Trino ; gateway autorise le dataset du chemin.

COMPOTEMENT CIBLE PROPOSÉ

Audit/Query Rust fournit un catalogue de templates versionnés, paramètres typés et sémantique testable. Toutes les relations du script doivent être liées et autorisées ; une règle monodataset refuse les autres, une règle multi-entrées les déclare explicitement. Les essais sont bornés et distincts des preuves de publication.

MIGRATION DE CETTE CAPACITÉ

Traiter l'écart d'autorisation des relations supplémentaires avant de reproduire le parcours. Versionner les corrections des templates pour ne pas changer en silence les règles enregistrées. Rejouer les cas vide, null et limites numériques/temporelles.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un script qui joint une table non autorisée est refusé avant Trino. Chaque template donne le résultat annoncé sur des jeux connus, y compris table vide. Un résultat de plusieurs lignes ou colonnes retourne l'erreur qualité attendue sans être enregistré comme preuve.

Références de l'analyse

monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataquality/aggregate/Template.kt:7 Les quinze modèles disponibles.

monorepo · services/data-serve/app/data-serve-domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataquality/aggregate/Template.kt:101 Opérateurs codés de maximum et médiane.

monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataquality/service/QualityRuleServiceImpl.kt:22 Le contrôle vérifie la présence de la cible, pas l'exclusivité des relations.

monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataquality/service/QualityRuleServiceImpl.kt:49 Forme du résultat exigée.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataquality/query/RuleCheckQueryHandler.kt:31](#) Exécution interactive.

DS-15 **Cible non qualifiée**

Calculer la qualité après les traitements et diffuser les résultats

COMPORTEMENT OBSERVÉ

Les événements process-data déclenchent CreateCompute. Le handler ignore les exécutions échouées et les branches autres que main. Il crée un Compute lié au dataset et à l'exécution, charge la dernière version de chaque règle active, valide et exécute son SQL, puis produit ComputedRule avec résultat ou erreur et publie un événement pour data-model. L'évaluation se fait sur la ref main résolue au moment de chaque requête, pas sur un snapshot fixé par l'événement de fin de traitement. Le handler présenté ne montre pas de déduplication de commande par executionId avant allocation d'un nouvel ID de compute.

UTILISATEURS ET RÔLES

Consommateur d'exécution, responsable qualité et notifications de résultat.

INTERFACES DANS LE CODE

ProcessDataEventConsumer ; CreateComputeCommandHandler ; ComputedRuleCreatedEvent.

DONNÉES ET ÉTATS

Compute, dernière RuleHistory choisie, ComputedRule et statut SUCCESS/FAIL/ERROR/PENDING ; publication outbox.

DÉPENDANCES ET ÉCHANGES

process-data Kafka, Trino, repositories de règles et topic data-serve-computed-rule vers data-model.

COMPORTEMENT CIBLE PROPOSÉ

Audit Rust évalue toutes les règles retenues pour un candidat précis avec un ensemble de versions figé. La preuve post-candidat est immuable et liée durablement à la provenance préallouée. Publisher consomme cette preuve avant d'approuver ; une alerte métier peut être non bloquante selon politique explicite. Le run d'audit est borné et idempotent.

MIGRATION DE CETTE CAPACITÉ

Importer les résultats historiques comme observations sans prétendre qu'ils ont bloqué la publication. Introduire clé candidat+ruleset+audit-policy et binding de snapshot, puis déplacer le déclenchement avant Publisher. Supprimer la dépendance au simple événement de succès de main.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un second traitement publié pendant l'audit ne change pas les données auditées. Rejouer le même message retrouve le même compute et la même preuve. Un veto garde l'ancien snapshot publié et interdit l'ack source. Tous les participants d'une release multi-sorties sont couverts.

Références de l'analyse

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataquality/command/CreateComputeCommandHandler.kt:122](#) Conditions de déclenchement et versions de règles.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataquality/command/CreateComputeCommandHandler.kt:58](#) Exécution sur main et construction des résultats.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/dataquality/event/consumer/ProcessDataEventConsumer.kt:19](#) Source événementielle du calcul.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/dataquality/event/producer/ComputedRuleEventHandlerKafkaImpl.kt:1](#)

Production des résultats de règles.

DS-16 **Cible non qualifiée**

Réparer la projection des types depuis le schéma physique

COMPORTEMENT OBSERVÉ

Une migration parcourt les datasets et branches, lit leur schéma physique par Trino, puis compare les types par nom de colonne sans distinction de casse. Elle conserve l'ordre et la casse déclarés et ignore les colonnes absentes physiquement. Si la lecture de branche échoue, elle essaie main. Elle émet une correction JSON vers data-model seulement lorsqu'un type diffère. Les exceptions d'une branche sont journalisées et le parcours continue. Cette correction passe encore par un producteur Kafka direct, exception au mécanisme outbox.

UTILISATEURS ET RÔLES

Opérateur de migration et mainteneur de la cohérence du catalogue.

INTERFACES DANS LE CODE

CorrectSchemaCommandHandler ; DatasetSchemaMigrationObject ; topic migration-topic-dataset-branch-schema. Aucune route REST publique de correction trouvée.

DONNÉES ET ÉTATS

Projection de types corrigée par événement ; schéma physique lu via snapshot/ref.

DÉPENDANCES ET ÉCHANGES

Trino/Iceberg, data-model consommateur de migration, runner de code migrations.

COMPORTEMENT CIBLE PROPOSÉ

Migration/Datasets Rust compare field IDs, schéma d'édition et annotations sans remplacer une branche inconnue par main. Le plan de réparation est audité, rejouable et sépare correction d'index de changement réel de données. Il ne publie pas automatiquement un nouveau snapshot.

MIGRATION DE CETTE CAPACITÉ

Utiliser la lecture physique pour l'inventaire initial, mais enregistrer UUID, snapshot et raison de l'écart. Remplacer le fallback implicite par un statut à résoudre. Préserver les anciens événements le temps de drainer leurs corrections.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une relance sans divergence n'émet aucune correction. Une branche inexistante est signalée sans lui attribuer le schéma de main. Un renommage de colonne conserve son field ID et n'est pas confondu avec une nouvelle colonne.

Références de l'analyse

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/command/CorrectSchemaCommandHandler.kt:33](#) Comparaison par noms et conservation de l'ordre.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/command/CorrectSchemaCommandHandler.kt:116](#) Fallback main et correction conditionnelle.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/event/CorrectSchemaEventHandlerKafkaImpl.kt:1](#) Producteur historique direct de correction.

DS-17 Cible non qualifiée**Borner les ressources et restaurer les projections après panne****COMPOTEMENT OBSERVÉ**

Les lectures interactives et dumps ont des pools Trino séparés. Les dumps utilisent `StreamingResponseBody` et une durée maximale MVC configurée. Le service utilise une outbox durable, un relay avec retries et purge des envois, des dead letters et un drainage qui clôt certaines courses où l'entité a disparu tout en signalant les erreurs permanentes. Le runner de migrations de code consigne les résultats mais appelle encore une sortie Spring de code zéro en cas d'échec et contient une réserve explicite sur l'absence de lock. Aucun cache de résultats SQL n'a été trouvé dans les implémentations lues ; seuls les caches d'identité/token et les projections sont observés.

UTILISATEURS ET RÔLES

Exploitant, SRE et clients soumis aux limites de concurrence.

INTERFACES DANS LE CODE

TrinoConfiguration ; OutboxRelay ; KafkaConfiguration ; DeadLetterEventDrainer ; CleyropMigrationRunner ; endpoints santé/metrics selon configuration.

DONNÉES ET ÉTATS

Pools et flux transitoires ; outbox_event, dead_letter_event et cleyrop_migration durables ; caches locaux reconstructibles.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, Kafka, Trino, S3 et IAM ; chart Deployment/HPA/PVC/NetworkPolicy.

COMPOTEMENT CIBLE PROPOSÉ

Query Rust possède admission, budget CPU/mémoire/octets, annulation et échéances par requête/run. Les workers sont fongibles. Tout cache de données est une optimisation indexée par tenant, édition, schéma, requête normalisée et contexte de sécurité, avec réévaluation des droits. Le journal durable sans JVM et les reçus remplacent la dépendance finale à Kafka Java.

MIGRATION DE CETTE CAPACITÉ

Établir des budgets d'exploitation mesurés et conserver l'isolement des téléchargements. Drainer outbox/dead letters avant coupure et qualifier les duplicates. Ajouter un lock et un statut d'échec observable aux migrations Rust. Tester restauration de PG, index et objets séparément.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une panne broker n'acquiesce pas une mutation non journalisée. Une migration échouée empêche readiness et fournit une erreur identifiable. Supprimer un cache ne provoque ni nouvelle publication ni nouvel export. Un utilisateur révoqué ne récupère pas un résultat d'un cache partagé.

Références de l'analyse

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/configuration/TrinoConfiguration.kt:17](#) Pools et timeouts distincts.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/common/outbox/OutboxPublisher.kt:9](#) Transactions et garanties au moins une fois.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/common/deadletter/DeadLetterEventDrainer.kt:25](#) Cycle des dead letters et backlog.

[monorepo · services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/codemigration/runner/CleyropMigrationRunner.kt:29](#) Comportement d'échec et réserve sur lock.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/authz/KeycloakServiceTokenProvider.kt:24](#) Cache de token local observé.

DS-18 **Cible non qualifiée**

Distinguer API de lecture, exposition déclarée et export durable

COMPORTEMENT OBSERVÉ

ExpositionApiController est aujourd'hui une famille d'API OAuth de listing, lecture, métadonnées, SQL et téléchargement. Aucun objet EXPOSURE déclarant consommateur, responsable, politique de refresh ou édition cible, ni ressource EXPORT avec destination, état et reçu, n'a été trouvé dans les contrôleurs et agrégats examinés. Il n'y a pas non plus de gestion locale de tokens API dans ces contrôleurs. Les téléchargements restent des requêtes HTTP transitoires. Cette fiche spécifie le besoin cible et évite d'attribuer au nom du contrôleur une capacité absente.

UTILISATEURS ET RÔLES

Application, dashboard, intégration et responsable d'un consommateur externe.

INTERFACES DANS LE CODE

Existant : routes ExpositionApiController. API cible EXPOSURE/ExportEffect à définir sans inventer de chemin compatible.

DONNÉES ET ÉTATS

Cible : consommateur déclaré en PostgreSQL, EditionRef source, demande d'effet, clé d'idempotence et reçu ; pas de neuvième kind EXPORT.

DÉPENDANCES ET ÉCHANGES

Catalog, Query, Applications, Egress, IAM et Audit ; destination externe validée.

COMPORTEMENT CIBLE PROPOSÉ

Applications/Catalog Rust décrivent EXPOSURE comme consommateur déclaré d'éditions. Egress traite EXPORT comme effet consommant une édition publiée avec destination, contrat de format, autorisation, preuve de livraison et politique de reprise. MODEL/EMBEDDINGS peuvent avoir un alias de serving sans déplacer l'autorité de leur édition Iceberg.

MIGRATION DE CETTE CAPACITÉ

Inventorier clients OAuth/MCP, dashboards et téléchargements programmés, puis les rattacher à des consommateurs déclarés. Garder l'API native pour les clients existants. Migrer les effets vers des jobs bornés qui ne relisent pas implicitement main après retry.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une demande d'export sur un candidat non publié est refusée. Une livraison dont la réponse a été perdue retrouve son reçu ou déclenche une réconciliation, sans doublon silencieux. Une exposition épinglée reste sur son édition jusqu'à une mise à jour autorisée.

Références de l'analyse

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[app/src/main/kotlin/com/cleyrop/data/serve/app/apitoken/controller/ExpositionApiController.kt:69](#) Famille de routes de lecture, sans entité de consommateur.

[monorepo](#) · [services/data-serve/app/data-serve-](#)

[domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/query/DownloadDatasetQueryHandler.kt:44](#) Téléchargement transitoire, sans reçu d'effet.

monorepo · services/data-serve/app/data-serve-

domain/src/main/kotlin/com/cleyrop/data/serve/domain/dataset/aggregate/Dataset.kt:128 Modèle de serving observé.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Conserver les façades REST nécessaires, les définitions et versions de règles, les déclarations EXPOSURE, commandes d'export et reçus d'effet. Leur stockage PG ne remplace pas les éditions publiées ni leur checkpoint.
- Créer ReadSet et QuerySession avec sujet/acteur, tenant, édition et schema de chaque entrée, politique, paramètres, budget et échéance. La pagination reprend ce contrat et ne résout pas une nouvelle version de main.
- Exprimer les capacités SQL du moteur sans JVM et refuser les plans non pris en charge avant exécution. Une interface pgwire/Flight SQL n'est pas une preuve de compatibilité avec tous les clients Trino ou Postgres.
- Versionner les règles qualité et publier un manifeste de preuve attaché au candidat. Historique utilisateur des publications et historique technique des snapshots restent distincts.

Plan de données proposé

- Icegres ou un autre moteur Rust qualifié exécute les scans/plans sur ReadSet ; Arrow sert de contrat interne typé. La sélection du moteur n'est pas qualifiée par cette lecture du service Kotlin.
- Séparer budgets et admission des lectures interactives, checks qualité, previews, exports et scans de migration. Annulation et deadline doivent libérer objets, streams et slots.
- Les caches sont restructurables et identifiés par édition/plan/schéma/tenant ; les droits sont réévalués. Un index de serving, d'embeddings ou un alias d'application ne devient pas autorité de publication.
- Egress produit CSV, NDJSON ou formats nouveaux depuis une édition publiée et livre via des jobs bornés. Les accusés de livraison et reprises sont des reçus d'effets, séparés de l'ack de source.
- Audit exécute sur candidat immuable et lié par provenance durable. Les workers ne reçoivent ni credential global de table ni droit de commit/drop ; Writer et Publisher restent les seuls médiateurs d'écriture et publication.

Découpage du core Rust

- Proposition query-api : routes compatibles, enveloppes d'erreurs, parsing des paramètres, authentification IAM et propagation du contexte MCP.
- Proposition query-planner : SQL AST, binding dataset/field IDs, résolution du ReadSet, validation des fonctions et règles d'accès sur l'ensemble du plan.
- Proposition query-engine-adapter : interface vers Icegres/moteur sans JVM, Arrow, annulation, budgets, restitution des types et métriques.
- Proposition query-pages : curseurs signés liés au ReadSet et au sujet, total cohérent, tri stable et réautorisation.
- Proposition egress-codecs : CSV compatible, NDJSON schema/row/end, checksum et formats typés futurs avec versions explicites.

- Proposition audit-rules : templates, règles versionnées, comparateurs exacts, exécution sur candidat et preuves immuables ; migration des scripts SQL historiques.
- Proposition serving-projector : consommation dédoublée des événements de catalogue, contrôle de révisions et reconstruction des miroirs.

Identité, autorisation et isolation

- La décision du gateway ne suffit pas pour les datasets nommés dans un SQL ou un scope preview. Le code délègue ces décisions à IAM et échoue fermé sur réponse invalide ; conserver ce comportement.
- Les routes de règles vérifient le lien ruleId/datasetId. En revanche, QualityRuleServiceImpl vérifie seulement que le dataset ciblé est présent dans les relations SQL. Qualifier puis corriger le traitement des relations supplémentaires avant migration.
- Les identités gateway, OAuth natif et MCP sont trois transports à préserver. Les headers de compte technique ne doivent pas remplacer le sujet utilisateur. Les chemins directs au pod nécessitent des règles réseau compatibles avec leur identité.
- La ref physique résolue pour une requête n'est pas encore une preuve de publication approuvée. L'API future doit empêcher lecture des candidats, snapshots techniques et fichiers d'audit non autorisés.
- Les logs observés contiennent des statements SQL ou fragments rejetés ; un programme de portage doit définir expurgation, rétention et accès car ils peuvent contenir données personnelles ou literals sensibles.
- Le service n'expose pas de tenantId universel dans ses agrégats. Un déploiement Rust mutualisé doit rendre le tenant explicite dans les décisions, pools, caches, sessions et destinations d'export.

Exploitation et reprise

- Chart Helm de Deployment avec HPA et PVC optionnels, probes de santé et NetworkPolicy configurable. La politique réseau n'est pas une preuve qu'un environnement l'a activée ; elle doit couvrir gateway, MCP direct, IAM, PostgreSQL, Kafka, Trino et S3 selon le déploiement.
- Le pool de téléchargement est indépendant du pool interactif. Le délai MVC borne les dumps ; la saturation du pool de download ne doit pas consommer tous les slots de lecture. Aucun chiffre de charge ni SLO n'est qualifié ici.
- Les événements sortants passent par outbox sauf la correction de schéma historique. La durabilité de l'insert outbox ne suffit pas à prouver une transaction commune avec toutes les écritures du handler ; vérifier les frontières lors du portage.
- KafkaConfiguration traite retry, dead letters et repli local ; DeadLetterEventDrainer distingue les erreurs permanentes des courses d'entité disparue et expose des métriques de backlog. Préserver un traitement opérable au remplacement du journal.
- Flyway et code migrations se lancent au démarrage. Le runner data-serve présente encore un chemin de sortie zéro sur échec et un TODO de verrouillage. La cible doit empêcher deux instances de lancer la même migration mutable.
- Sauvegarder règles et histories, commandes/reçus d'effets, preuves et mappings d'identité. Reconstruire projections et caches depuis les sources autoritaires sans rejouer les téléchargements ou exporter une deuxième fois.

Séquence de migration

- Inventorier les clients des API natives, éditeur, previews, SDK et MCP ; documenter leurs formats, SQL, limites, erreurs et modes d'identité. Associer chaque dataset à une table UUID et à une base publiée retenue.
- Construire une façade Rust en lecture sur les mêmes projections et ReadSets autorisés. Comparer métadonnées, filtres, previews, paginations et exports sur des fixtures expurgées.
- Qualifier le moteur sans JVM pour SQL, types, fonctions, timezone, nulls, decimal, binaire et structs. Vérifier EXPLAIN, annulation, dépassements de mémoire et garanties d'isolation. Aucun retrait de Trino avant ces critères.
- Introduire éditions épinglées et sessions de pagination, avec tests de publication et révocation concurrentes. Corriger l'analyse des règles et les écarts de templates avant d'en faire des gates WAP.
- Migrer Rule/RuleHistory et calculs vers Audit ; figer versions et candidats. Basculer de l'événement post-traitement vers preuve avant publication, avec idempotence par candidat/ruleset.
- Déplacer les téléchargements lourds et effets externes vers Egress, reçus et runs bornés. Drainer outbox/dead letters et l'ancien calcul qualité pour éviter deux producteurs.
- Retirer l'ancien service, Trino, JDBC, JSqlParser, bibliothèques Spring et dépendances Keycloak/Kafka Java après bascule de toutes les cohortes et exercice de restauration.

Écarts et décisions encore ouverts

- Lecture statique uniquement. Aucun benchmark, requête Trino/Icegres, test de contrat ni restauration n'a été exécuté pour ce rapport.
- Quel dialecte SQL est contractuel pour les clients existants, et quelles fonctions ou types doivent être portés, traduits ou refusés explicitement par le moteur Rust ?
- Quelle garantie fournir aux requêtes multi-tables hors release commune ? La résolution indépendante de refs actuelle ne prouve aucune visibilité atomique de groupe.
- Quel protocole d'édition et de curseur ajouter à la pagination numérique pour conserver une lecture stable sans rompre les intégrations ? Que faire d'une révocation au milieu d'un stream déjà ouvert ?
- Le contrôle SQL de qualité doit-il devenir strictement monodataset ou supporter des règles multi-entrées explicitement autorisées ? Le code actuel ne vérifie pas toutes les relations de ce parcours.
- Comment versionner la correction des templates maximum/médiane et l'adoption de nombres exacts sans modifier les preuves et règles historiques en place ?
- Quels computes ont été réellement évalués sur l'état correspondant à leur executionId ? Le handler lit main au moment de chaque règle et ne porte pas de snapshot explicite.
- Quelle rétention et quels droits appliquer aux SQL, erreurs moteur et previews objets ? Les sources peuvent contenir des données sensibles même sans credentials.
- Le cache de données n'est pas une capacité constatée du service actuel. Sa conception future doit spécifier clés, invalidation, révocation et coûts avant toute promesse de performance.
- EXPOSURE et EXPORT nécessitent de nouveaux contrats. L'API nommée ExpositionApiController ne possède pas aujourd'hui ce cycle de vie de consommateur ou d'effet.

Références de l'analyse

[monorepo](#) · [services/data-serve/AGENTS.md:1](#) Périmètre et contraintes du composant.

[monorepo](#) · [services/data-serve/app/gradle/libs.versions.toml:20](#) Versions Kotlin/Spring déclarées.

[monorepo](#) · [services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataset/controller/ServeController.kt:55](#) Aperçu, droits, SQL et téléchargements UI.

[monorepo](#) · [services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/apitoken/controller/ExpositionApiController.kt:182](#) Exposition native et MCP.

[monorepo](#) · [services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/dataquality/controller/DataQualityController.kt:83](#) Contrats qualité publics.

[monorepo](#) · [services/data-serve/chart/values.yaml:147](#) Bascule du miroir d'identité dans le chart.

[monorepo](#) · [services/data-serve/chart/values.yaml:353](#) Isolation réseau configurable.

[monorepo](#) · [services/data-serve/app/data-serve-app/src/main/kotlin/com/cleyrop/data/serve/app/codemigration/runner/CleyropMigrationRunner.kt:29](#) Cycle de migration observé.